MCU TINKERING LAB

robocar-unified

Single-Board AI Robot Car

Assembly, Wiring & Firmware Build Guide

A hands-on guide to assembling the consolidated robocar on a **Seeed Studio XIAO ESP32-S3 Sense**. Camera capture, Gemini AI planning, motor control, and peripherals all run on one module.

Target MCU	ESP32-S3 (8 MB PSRAM / flash)
Toolchain	ESP-IDF v5.4 (containerized)
Difficulty	Intermediate · ~2–3 h

Pin data auto-generated from `main/pin_config.h` via `just robocar-unified::gen-pin-defs`.
All components must share a common ground.

Contents

1 · Overview 3
2 · Bill of Materials 3
Tools required 4
3 · System Architecture 4
4 · Wiring Reference 5
4.1 · XIAO ESP32-S3 GPIO assignments 5
4.2 · I²C topology (TCA9548A @ 0x70) 5
4.3 · PCA9685 channel map (0x40, 100 Hz) 5
4.4 · Ultrasonic rangefinders (2× RCWL-1601, I²C) 6
4.5 · Audio output (MAX98357A I²S amplifier) 6
4.6 · Board pinouts 7
5 · Power 9
Suggested capacitors 10
6 · Assembly Steps 10
7 · Build & Flash the Firmware 11
8 · First Boot & Provisioning 11
8.1 · WiFi over the USB console (Improv Serial) 11
8.2 · Discovery & AI backend 11
8.3 · Over-the-air updates 12
9 · Functional Checkout 12
10 · Troubleshooting 12

1 · Overview

The **robocar-unified** project consolidates the original dual-ESP32 robocar (a Heltec main controller plus an ESP32-CAM) onto a single **XIAO ESP32-S3 Sense**. One module now handles camera capture, AI inference, motor control, and all peripherals, using dual-core affinity to keep motor timing isolated from bursty network and vision work.

The control system is **hierarchical**: a slow **planner** (Core 1) calls Google's Gemini Robotics-ER to emit structured goals, and a fast **reactive executor** (~30 Hz, Core 0) drives the robot smoothly toward those goals while a front ultrasonic rangefinder provides an independent obstacle reflex. The planner is not on a fixed schedule: the board boots **dormant** and asks Gemini only when something happened, backing off 15 s → 300 s when nothing does.

Core 0 — real-time
Reactive executor (visual servo, heading hold, motor PWM), peripheral I/O, command console, and the front rangefinder's obstacle reflex.

Core 1 — bursty I/O
Planner (Gemini calls), OV3660 camera DMA, audio, WiFi / MQTT / OTA.

What you get
A two-wheel-drive car that captures frames, asks Gemini what to do, and drives toward goals — with pan/tilt camera, status LEDs, an OLED display, buzzer feedback, WiFi provisioning over the USB console, and over-the-air updates.

2 · Bill of Materials

Qty	Component	Notes
1	XIAO ESP32-S3 Sense	MCU + OV3660 camera + PDM mic + 8 MB PSRAM, USB-C
1	TCA9548A I²C multiplexer	Breakout, address 0x70
1	PCA9685 16-ch PWM driver	Breakout, address 0x40
1	TB6612FNG dual motor driver	Breakout
1	SSD1306 OLED display	128×64, I²C, address 0x3C
1	MCP23017 GPIO expander	Breakout, address 0x20 — optional
1	MAX98357A I²S class-D amplifier	Mono, for voice output
1	Speaker	4–8 Ω, 2–3 W
2	RCWL-1601 ultrasonic rangefinder	Front + rear; I²C mode: 10 kΩ on R4, R5 empty
2	10 kΩ chip resistor	For R4 on each rangefinder; size it to the pad
4	10 kΩ resistor	I²C pull-ups to 3.3 V on mux ch3 and ch4, see §4.4
2	RGB LED	Common-anode
2	SG90 micro servo	Pan / tilt
2	DC gear motor + wheel	~3–6 V hobby motors
1	Piezo buzzer	Passive
1	100 Ω resistor	In series with buzzer
3	Electrolytic capacitor	470 µF, 16 V — C1–C3, see §5
3	Ceramic capacitor	100 nF — C4–C6, optional, see §5
2	18650 Li-ion cell + holder	Wired in SERIES — 7.4 V nominal
1	LM2596 buck converter module	Adjustable — set to 5.0 V

Qty	Component	Notes
—	Chassis, wiring, headers	2WD car chassis, jumper wires, standoffs

Tools required

Soldering iron + solder, wire strippers, small screwdriver set, multimeter (for verifying 5 V rail and continuity), a USB-C cable, and a computer with Docker (for the containerized firmware build).

> **Rangefinder mode — read this**
> The RCWL-1601s ship in trigger/echo mode. Before wiring, solder a **10 kΩ resistor on R4** (`IIC`) on each and leave R5 (`UART`) empty: the module then answers on I²C at 0x57. Without it the module never answers and the `sonar` bring-up check reports it absent. Run them from 3.3 V.

3 · System Architecture

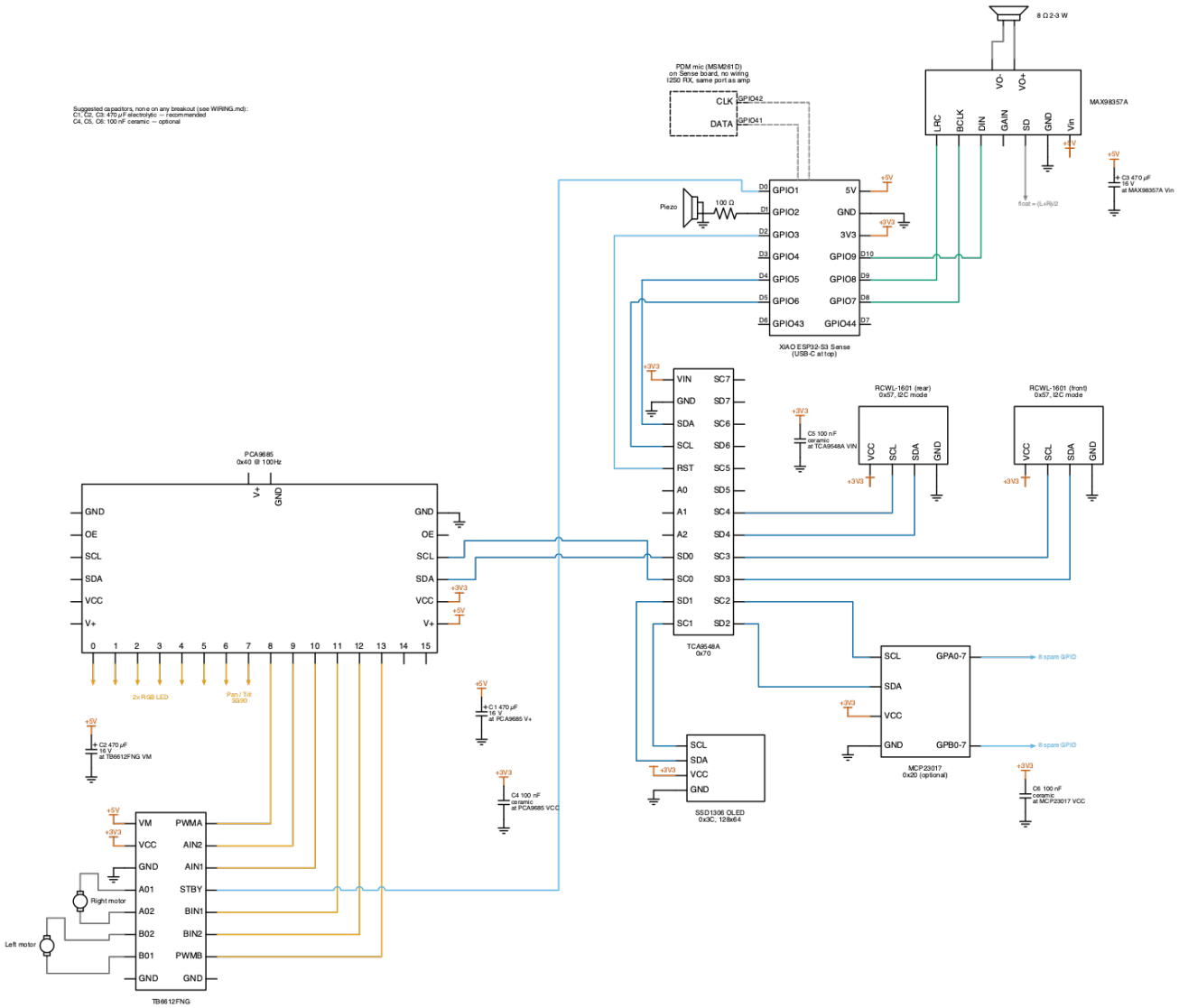


Figure 1: Full wiring schematic. Source: `docs/schematics/circuits/robocar_unified.py`.

Everything on the I²C bus hangs off the **TCA9548A multiplexer** — the ESP32-S3 never talks to the PCA9685 or OLED directly. This lets devices that would otherwise share addresses coexist — the two rangefinders both answer at 0x57 — and keeps the two 400 kHz devices on separate channels. The ESP32-S3 itself drives only five groups of

signals directly: I²C (GPIO5/6) and the multiplexer's reset (GPIO3), the motor-enable STBY line (GPIO1), the buzzer (GPIO2), and **I²S audio** (GPIO7/8/9) to the MAX98357A amplifier.

4 · Wiring Reference

4.1 · XIAO ESP32-S3 GPIO assignments

Only 11 GPIOs are exposed on the XIAO headers. Camera pins are internal to the Sense module and do not conflict.

XIAO Pin	GPIO	Function	Notes
D0	GPIO1	TB6612FNG STBY	HIGH = motors enabled
D1	GPIO2	Piezo buzzer	LEDC PWM · 100 Ω in series
D2	GPIO3	TCA9548A RST	Active low; pulsed to free a stuck bus
D3	—	*spare*	Free since the rangefinders moved to I²C
D4	GPIO5	**I²C SDA**	to TCA9548A
D5	GPIO6	**I²C SCL**	to TCA9548A
D6	GPIO43	UART0 TX	Spare · ROM boot log at reset
D7	GPIO44	UART0 RX	Spare · console is on USB-C
D8	GPIO7	**I²S BCLK**	to MAX98357A — bit clock
D9	GPIO8	**I²S LRCLK**	to MAX98357A — word select
D10	GPIO9	**I²S DIN**	to MAX98357A — serial data

I²C runs at **400 kHz**.

> **Three spare pads: D3, D6/D7**
> D3 and D6/D7 (GPIO43/44, UART0) are the only free header pads. The ROM bootloader prints its boot log on GPIO43 at every reset, so anything wired to D6 sees that traffic. Beyond those, digital I/O goes through the MCP23017 on TCA9548A channel 2.

4.2 · I²C topology (TCA9548A @ 0x70)

Select the channel on the multiplexer **before** addressing any downstream device. The MCP23017 on ch2 is optional — the firmware boots fine without it.

Channel	Device	Address
ch0	PCA9685 PWM driver (motors, servos, LEDs)	0x40 @ 100 Hz
ch1	SSD1306 OLED display (128×64)	0x3C
ch2	MCP23017 GPIO expander (optional)	0x20
ch3	RCWL-1601 rangefinder, front (fixed)	0x57
ch4	RCWL-1601 rangefinder, rear	0x57
ch5–7	*reserved — IMU / ToF / future sensors*	—

4.3 · PCA9685 channel map (0x40, 100 Hz)

All motor direction, motor PWM, servo, and LED outputs go through the PCA9685. Motor direction pins use PCA9685 "full-on" (4096) / "full-off" (0); PWM pins use the full 12-bit range (0–4095).

Ch	Signal
0	Left LED — R
1	Left LED — G
2	Left LED — B
3	Right LED — R
4	Right LED — G
5	Right LED — B
6	Pan servo (SG90)
7	Tilt servo (SG90)

Ch	Signal
8	Motor R — PWM → PWMA
9	Motor R — IN2 (dir) → AIN2
10	Motor R — IN1 (dir) → AIN1
11	Motor L — IN1 (dir) → BIN1
12	Motor L — IN2 (dir) → BIN2
13	Motor L — PWM → PWMB
14–15	*reserved*

100 Hz is a chip-wide compromise: one prescaler serves the servos, the motors and the LEDs. Measured on this build — the SG90s fitted track at 50, 100 and 125 Hz and buzz at 200, so this sits one rung below the highest rate that worked. Raise it for steadier LEDs, lower it for happier servos; `servo freq <hz>` retunes it live, without a reflash.

4.4 · Ultrasonic rangefinders (2× RCWL-1601, I²C)

Both modules run in I²C mode at 0x57, each alone on its own TCA9548A channel: the address is fixed, and the module "doesn't like to share the I²C bus" (Adafruit #4742). In I²C mode the header's `Trig` pin is SCL and `Echo` is SDA.

Pin	Front (ch3)	Rear (ch4)
VCC	3.3 V	3.3 V
SCL (`Trig`)	TCA9548A SC3	TCA9548A SC4
SDA (`Echo`)	TCA9548A SD3	TCA9548A SD4
GND	any GND	any GND

Fit 10 kΩ pull-ups to 3.3 V on all four lines (SC/SD of both channels): the modules have none, and the TCA9548A breakout has none on its downstream channels either. A reading takes ~120 ms, so each sensor reports about every 150 ms; the firmware never holds the bus while one converts. **Obstacle reflex:** if the front distance drops below 15 cm, the executor brakes, independent of planner goals. The rear sensor is reported in telemetry only.

4.5 · Audio output (MAX98357A I²S amplifier)

The robot speaks through a mono I²S class-D amplifier. Audio is 24 kHz 16-bit PCM — the native output rate of the Gemini TTS model — decoded incrementally into a PSRAM ring so playback starts before the download finishes.

Signal	Pin	Function
BCLK	GPIO7 (D8)	Bit clock
LRCLK	GPIO8 (D9)	Word select / left-right clock
DIN	GPIO9 (D10)	Serial audio data to amplifier
Vin	5 V	Power — see §5 for supply requirements
GND	any GND	Shared ground
SD_MODE	floating	(L+R)/2 — firmware duplicates mono into both slots
GAIN	float	9 dB default

The I²S channel is disabled between utterances — the MAX98357A hisses faintly whenever BCLK is running, so leaving it clocking silence is audible.

> **Trade-off**
> Wiring the amplifier on D8–D10 **replaces the microSD slot**. On the Sense expansion board these three pins

> are the SD card's SPI bus. There is no alternative pin set — I²S needs a hardware peripheral, so it cannot be moved behind the PCA9685 or the MCP23017.

4.6 · Board pinouts

Every header pad of each board, on the edge it sits on and in the order the vendor's own board file gives — generated by `tools/hardware/pinout.py` from the pages in `docs/reference/boards/`, not drawn from a photo. Blue pads carry a signal from the XIAO or from a PCA9685 channel and are labelled with it; gold pads are power, the I²C chain between boards (see §4.2), or unused. The drawings are not to scale.

The MCP23017 and SSD1306 modules are not drawn: no vendor board file identifies the fitted modules, so their header order is unknown here (issue #662). Read the order off each module's own silkscreen. The RCWL-1601 is drawn, but from the listing's photos — no board file is published for it — and is not yet checked with a meter.

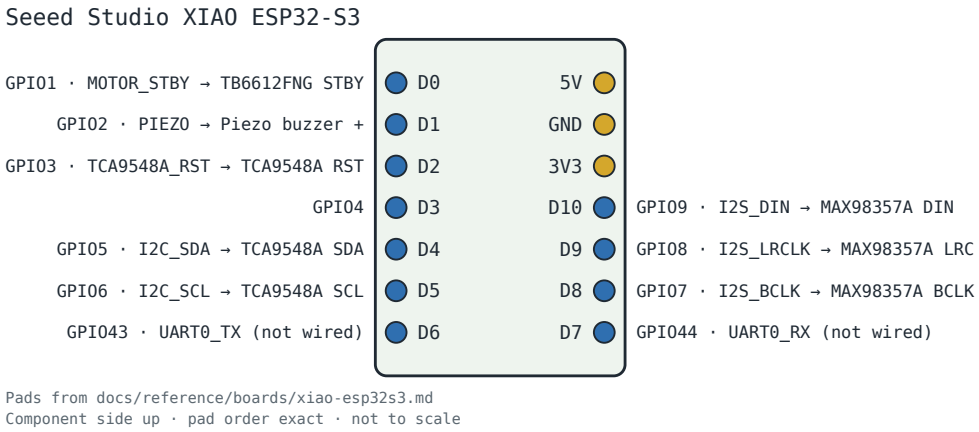


Figure 2: XIAO ESP32-S3, USB-C at the top. The Sense expansion board leaves this header unchanged.

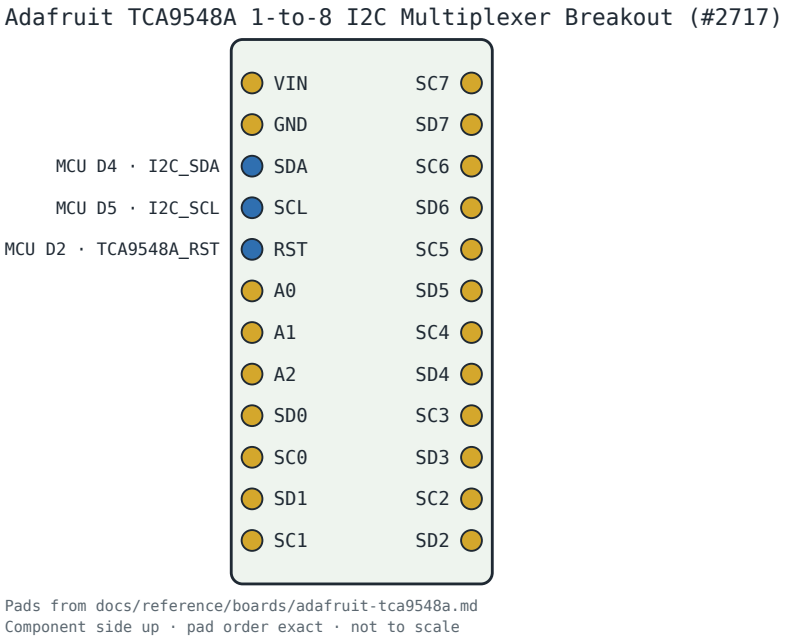


Figure 3: TCA9548A, `VIN` at the top-left corner. On the left edge each channel runs `SDn` above `SCn`; on the right, `SCn` above `SDn`.

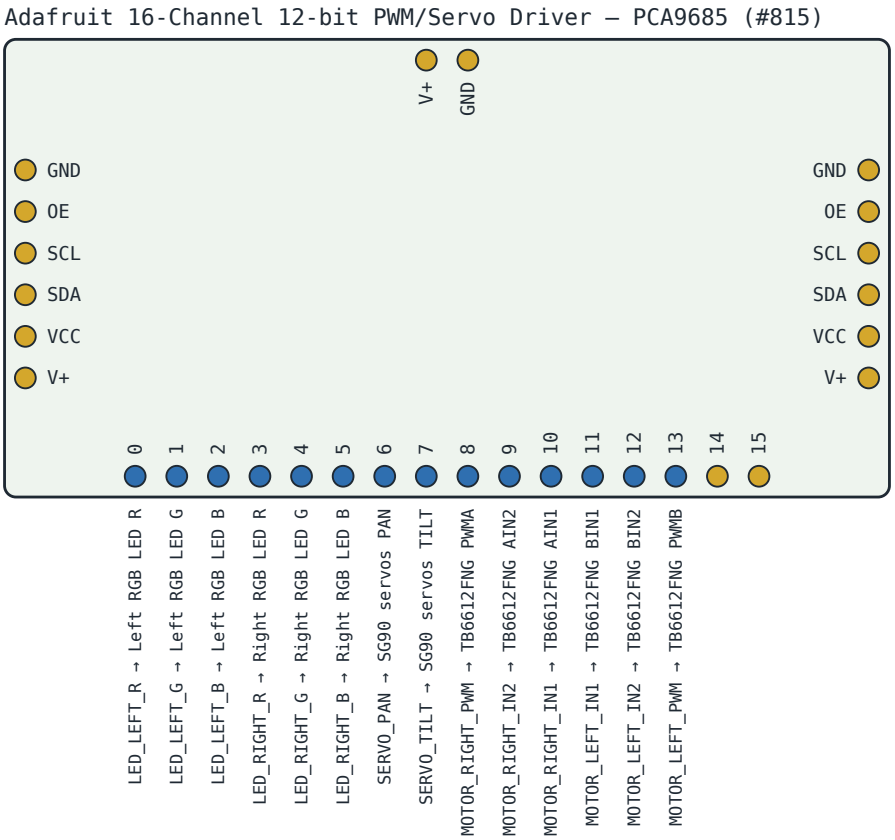


Figure 4: PCA9685, power terminal at the top. Only the `PWM` row of each servo column is drawn; behind it sit `V+`, then `GND` at the board edge. The two side headers carry the same six signals. Each driven channel is labelled with its role and the pin it reaches; the same map as a table: §4.3.

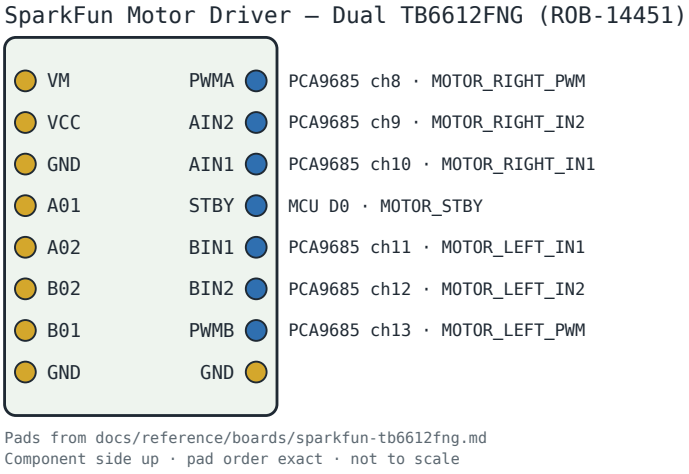


Figure 5: TB6612FNG, SparkFun logo at the top. The control row is driven from the PCA9685 (§4.3); only `STBY` comes from the XIAO.

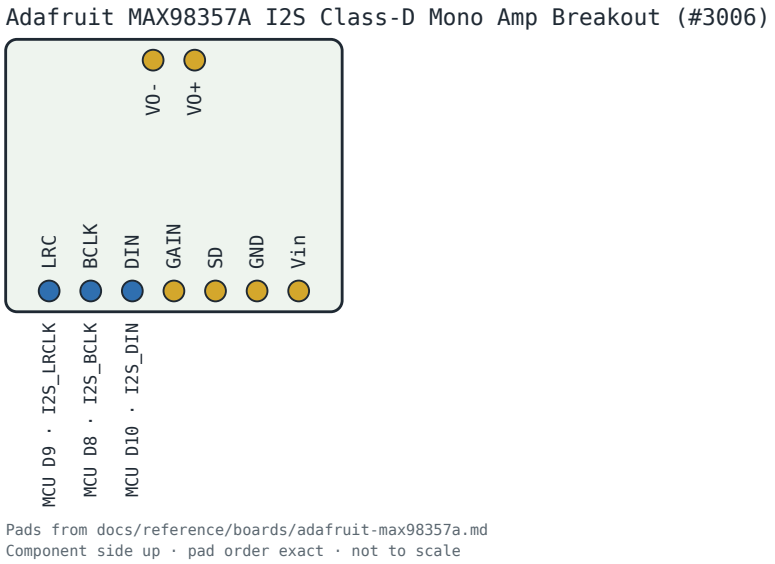


Figure 6: MAX98357A, speaker terminal at the top, the 7-pin header along the bottom.

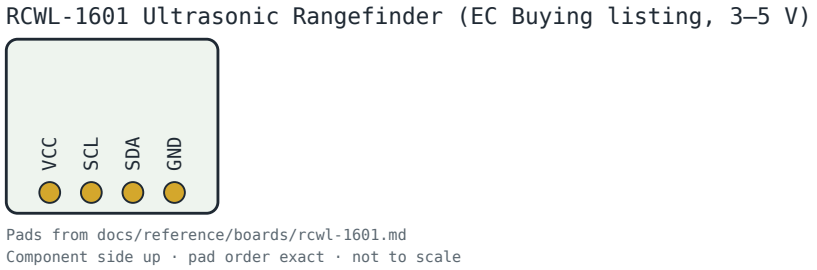


Figure 7: RCWL-1601, transducers facing you, the header on the bottom edge; front and rear are the same board. Read from the listing's silkscreen photos, not a board file — confirm `Gnd` and `Vcc` with a meter first.

5 · Power

Power the car from a **2×18650 pack wired in series** (7.4 V nominal, 8.4 V charged) through an **LM2596 buck converter set to 5.0 V**. Series and buck go together: a step-down regulator needs its input above its output, so a parallel 3.7 V pack could not feed it. Distribute that 5 V rail to the XIAO 5 V pin, the TB6612FNG's **VM**, the PCA9685's **V+**, and the **MAX98357A amplifier (Vin)**; the servos take their power from the PCA9685's V+ terminal.

The rail dies before the cells do: the LM2596 needs its input about **1.25 V above its output at 3 A** (0.95 V at 1 A), so 5 V out drops out near **6.3 V of pack** — roughly 3.15 V per cell, which a 2S pack reaches while it still has usable charge. The rail sags rather than shutting down cleanly, and nothing announces it. Weak servos, clipping audio and random resets are all plausible low-battery symptoms; measure the pack before diagnosing anything else on this rail.

The 3.3 V logic for the OLED, both rangefinders, TCA9548A, MCP23017, and the **VCC** pins of the TB6612FNG and

Golden rule

Common ground everywhere. Every module — the regulator, XIAO, motor driver, PCA9685, servos, sensors — must share one GND. Missing grounds cause brown-outs, I²C lockups, and erratic motion.

Brown-out detection is disabled in firmware because motor inrush was tripping it; a stiff 5 V supply and thick power wires matter.

PCA9685 comes from the XIAO's 3V3 pin. Those two VCC pins are logic supplies, not power inputs: both parts set their input-high threshold at 0.7 × their own VCC, so on 5 V it is 3.5 V — above what the XIAO's 3.3 V I²C and STBY lines drive, and the motors stay in standby. Keep motor/servo current (high, noisy) on the 5 V rail and logic on 3V3.

Amplifier supply — read this: The MAX98357A draws up to 1 A peaks into a 4 Ω load. With brown-out detection already disabled for motor inrush, an undersized rail will not warn you — it will present as random resets or corrupt audio mid-sentence. Fit a **470 µF bulk capacitor** at the amplifier's Vin (the breakout already carries the datasheet's 0.1 µF + 10 µF), and the same at the PCA9685's **V+** — the servos are the harsher transient source. Star-wire the rail: every load takes its own feed from the regulator's output terminal, because a servo's inrush travelling through the amplifier's feed wire is heard as distortion. Never power servos from the XIAO's 5 V pin or from USB. An **8 Ω speaker** roughly halves peak current versus 4 Ω.

Suggested capacitors

The schematic draws six capacitors that no breakout carries. Solder each one across the pin it names and the nearest GND pad, with short leads — a capacitor at the regulator's end of a jumper cannot supply a transient at the far end. Electrolytics go stripe (–) to GND. 16 V leaves margin even over a mis-set regulator, which cannot output more than the 8.4 V pack it steps down.

Ref	Part	Fit at	Status	Why
C1	470 µF electrolytic	PCA9685 V+	Recommended	Servo transients; the board's empty electrolytic footprint
C2	470 µF electrolytic	TB6612FNG VM	Recommended	Motor start/stall current; board fits only 10 µF + 0.1 µF
C3	470 µF electrolytic	MAX98357A Vin	Recommended	Datasheet asks for bulk on long supply runs; board fits 10 µF + 0.1 µF
C4	100 nF ceramic	PCA9685 VCC	Optional	Board fits only a 10 µF
C5	100 nF ceramic	TCA9548A VIN	Optional	Board fits only a 10 µF
C6	100 nF ceramic	MCP23017 VCC	Optional	Module unidentified; skip if it has one

Sources, board file by board file, are in `WIRING.md` § Suggested capacitors.

6 · Assembly Steps

1. **Mount the mechanics.** Fit the two gear motors and wheels to the chassis, add the caster/third wheel, and mount the battery holder low and centered.
2. **Wire power first.** Connect the series 18650 pack to the LM2596 input and, with nothing on its output, turn the trimpot until a multimeter reads **5.0 V**. The adjustable module spans 1.2–37 V and arrives set to anything; above 5.5 V damages the amplifier and the XIAO's regulator input. Only then run the 5 V and shared GND rails.
3. **Place the XIAO** and bring out I²C (GPIO5/6), the mux reset (GPIO3), STBY (GPIO1), buzzer (GPIO2), and I²S (GPIO7/8/9).
4. **Wire the I²C chain:** XIAO SDA/SCL ⟶ TCA9548A ⟶ PCA9685 (ch0), OLED (ch1), and optionally MCP23017 (ch2); XIAO D2 ⟶ TCA9548A RST. Pull-ups on the breakouts are usually sufficient.

5. **Wire the motor driver:** PCA9685 ch8–13 ⟶ TB6612FNG inputs; STBY ⟶ GPIO1; motor outputs ⟶ the two DC motors; VM ⟶ 5 V, VCC ⟶ 3V3. The PCA9685 likewise takes V+ from 5 V and VCC from 3V3.
6. **Add servos** (PCA9685 ch6/7) and **RGB LEDs** (ch0–5, common-anode).
7. **Add the buzzer** on GPIO2 through the 100 Ω resistor.
8. **Add the rangefinders** (R4 fitted, see §2): front on TCA9548A ch3, rear on ch4, 3.3 V power (§4.4).
9. **Wire the audio path:** MAX98357A BCLK/LRC/DIN ⟶ GPIO7/8/9, Vin ⟶ 5 V with C3 (470 µF) at Vin, speaker ⟶ amp output. Fit C1 at the PCA9685's V+ and C2 at the TB6612FNG's VM the same way. Leave SD_MODE floating for (L+R)/2.
10. **Double-check the 3.3 V vs 5 V rails** and confirm common ground with a multimeter continuity test before first power-up. TB6612FNG VM to VCC must read open; a beep means the 5 V and 3.3 V rails are shorted.

7 · Build & Flash the Firmware

Builds run inside a container — **no local ESP-IDF install is required**. The XIAO uses native USB-Serial-JTAG (USB VID `0x303a`), which `just` auto-detects.

```
# From the repo root
just robocar-unified::build                       # containerized ESP-IDF v5.4 build
PORT=/dev/cu.usbmodem* just robocar-unified::flash # flash over USB-C
just robocar-unified::monitor                      # serial console
# or, in one step:
just robocar-unified::flash-monitor
```

> **Can't enter download mode?**
> Hold **BOOT**, tap **RESET**, then release BOOT to force the bootloader, and re-run the flash command.

The flasher writes three images to an 8 MB, OTA-capable layout:

Offset	Image	Partition
`0x0`	`build/bootloader/bootloader.bin`	bootloader
`0x8000`	`build/partition_table/partition-table.bin`	partition table
`0x12000`	`build/robocar-unified.bin`	ota_0 (app)

8 · First Boot & Provisioning

8.1 · WiFi over the USB console (Improv Serial)

No WiFi credentials are compiled in. On first boot the device speaks **Improv Serial** on the same USB port you flashed it with – not the BLE variant, so there is nothing to pair. Open the board in a browser-based Improv Serial provisioner (Chrome desktop, e.g. ESP Web Tools) and send your SSID and password. They are written to NVS only after they are proven to connect, so a typo cannot overwrite a working network.

For local development you can instead copy `main/credentials.h.example` to `main/credentials.h` (gitignored) and hard-code credentials.

8.2 · Discovery & AI backend

After connecting, the car is reachable at **`robocar-unified.local`** via mDNS. The planner uses **Gemini Robotics-ER 2** to emit goals — `drive()`, `track()`, `rotate()`, and `stop()` — plus `speak()`, which renders a sentence through a second TTS model and plays it while the robot keeps driving.

8.3 · Over-the-air updates

OTA is enabled with app rollback. The updater pulls releases from the `laurigates/mcu-tinkering-lab` GitHub repo; `version.txt` is the single source of truth for the running version and is managed by release-please.

9 · Functional Checkout

Work through these after first flash, watching the serial monitor:

✓	Check	Expected result
☐	Boot log	No PSRAM / boot-loop errors; tasks pin to cores 0 & 1
☐	I²C scan	TCA9548A, PCA9685, and OLED all detected
☐	OLED	Status screen renders (128×64)
☐	LEDs	Both RGB LEDs cycle / show status colors
☐	Buzzer	Audible startup tone
☐	Servos	Pan/tilt center, then sweep within limits
☐	Motors	Both wheels drive forward and reverse; STBY HIGH
☐	Ultrasonic	Bring-up `sonar`: 0x57 answers on ch3 and ch4 readings track a hand
☐	Reflex	Car stops/reverses when an obstacle is < 15 cm
☐	Audio / TTS	Robot speaks startup message; speech queues without blocking motion
☐	Microphone	`mic` reports frames arriving, not `gate: DEAF`; `listen` answers a spoken question
☐	WiFi	Provisions via Improv; `robocar-unified.local` resolves

10 · Troubleshooting

Symptom	Likely cause & fix
Boot loop on power-up	Wrong PSRAM mode. The Sense uses **octal** PSRAM (`CONFIG_SPIRAM_MODE_OCT=y`); don't change it.
Random resets under motor load	Weak 5 V rail / missing common ground. Use thicker power wires and verify the LM2596 holds 5 V under load — scope it, a multimeter samples too slowly to see a millisecond sag.
Servos buzz but do not move	Almost always supply, not signal. Check V+ on the PCA9685 is fed from the regulator (VCC powers only the logic), and that the pack is above 6.3 V. `servo exercise` on the console logs every write, so a still servo with successful writes is a power fault.
No I²C devices found	Not selecting the TCA9548A channel first, or SDA/SCL swapped. Check GPIO5=SDA, GPIO6=SCL.
OLED and PCA9685 conflict	Both bypassing the mux. Route each through its own TCA9548A channel (ch1 / ch0).

Symptom	Likely cause & fix
Motors don't move	STBY (GPIO1) not HIGH, VM not on 5 V, or VCC on 5 V instead of 3V3 (STBY's threshold then sits above what a 3.3 V GPIO drives). Confirm TB6612FNG power and enable line.
Servos jitter or buzz	Shared noisy rail: keep servo power on 5 V with common ground. A servo that BUZZES and holds against a stop is the PWM frame rate, not the rail — these SG90s track at 50–125 Hz and stall at 200. `servo freq <hz>` retunes it live.
Board won't flash	Force download mode: hold BOOT, tap RESET, release BOOT.
Rangefinder never answers	Still in trigger/echo mode: fit 10 kΩ on R4, leave R5 empty. Or SCL/SDA swapped — in I²C mode `Trig` is SCL.
No audio / distorted speech	Weak MAX98357A supply. Fit ≥470 µF bulk cap at Vin, or use a separate 5 V feed. Check I2S wiring on GPIO7 – 9.

Authoritative pin data: `packages/robocar/unified/main/pin_config.h` · Design rationale: `docs/decisions/ADR-016-hierarchical-ai-controller.md`

Regenerate this PDF with `typst compile --root ../../../.. build-guide.typ`.